



Obra técnica de literatura especializada, produzida e diagramada conforme as normas ABNT para publicação editorial.

## Sumário

---

|                                                                    |    |
|--------------------------------------------------------------------|----|
| Guia: Replicando Token Economy via Git Submodules .....            | 4  |
| 1. Arquitetura: O que é Compartilhável vs. Específico .....        | 5  |
| Camada Global (Submodule) — Reutilizável em QUALQUER projeto . . . | 5  |
| Camada de Projeto (Fica no repositório do projeto) .....           | 6  |
| 2. Criando o Repositório Compartilhado .....                       | 6  |
| 2.1 Estrutura do Repositório token-economy-shared .....            | 6  |
| 2.2 Comandos para Criar .....                                      | 7  |
| 3. Adicionando o Submodule em Projetos Existentes .....            | 8  |
| 3.1 Comando Único .....                                            | 8  |
| 3.2 Estrutura Resultante .....                                     | 8  |
| 3.3 Script de Setup Automatizado .....                             | 8  |
| 4. Criando Projeto Novo com Token Economy .....                    | 10 |
| 4.1 Template de Projeto .....                                      | 10 |
| 4.2 Template CLAUDE.md Mínimo .....                                | 10 |
| 5. Sincronizando Atualizações .....                                | 11 |
| 5.1 Atualizar o Submodule no Projeto .....                         | 11 |
| 5.2 Propagar Atualização para Todos os Projetos .....              | 11 |
| 5.3 Script de Sincronização em Massa .....                         | 11 |
| 6. Submodule vs. Alternativas .....                                | 12 |
| 6.1 Git Submodule (Recomendado) .....                              | 12 |
| 6.2 Git Subtree .....                                              | 12 |
| 6.3 Package Manager (pip/npm) .....                                | 12 |
| 6.4 Copiar/Colar (Não Recomendado) .....                           | 12 |
| 7. Melhores Práticas .....                                         | 13 |
| 8. Exemplo Prático: Fabrica → Novo Projeto .....                   | 13 |
| 9. Conclusão .....                                                 | 14 |

## Guia: Replicando Token Economy via Git Submodules

---

**Data:** 2026-08-07 **Objetivo:** Reutilizar skills de compressão, MCPs, hooks, scripts e configs de economia de tokens em qualquer projeto novo ou existente.

---

# 1. Arquitetura: O que é Compartilhável vs. Específico

---

## Camada Global (Submodule) — Reutilizável em QUALQUER projeto

| Componente             | Caminho                                | Função                                   |
|------------------------|----------------------------------------|------------------------------------------|
| lean-ctx               | .claude/skills/lean-ctx/               | Economia de contexto: grep antes de read |
| headroom               | .claude/skills/headroom/               | Compressão de logs > 7 linhas            |
| caveman                | .claude/skills/caveman/                | Respostas telegráficas, sem enrolação    |
| rtk-memory             | .claude/skills/rtk-memory/             | Registro de erros e padrões              |
| pre-flight-check       | .claude/skills/pre-flight-check/       | Validação antes de commit/deploy         |
| calcular-gastos-sessao | .claude/skills/calcular-gastos-sessao/ | Cálculo de gastos por sessão             |
| fable-method           | .claude/skills/fable-method/           | Loop de resolução de problemas           |
| fable-judge            | .claude/skills/fable-judge/            | Verificação adversarial de trabalho      |
| self-learning          | .claude/skills/self-learning/          | Captura de golden paths                  |
| code-review-graph      | .code-review-graph/                    | Grafo de conhecimento do codebase        |
| pytest.ini             | pytest.ini                             | Configuração de testes                   |
| requirements.txt       | requirements.txt                       | Dependências Python                      |

Camada de Projeto (Fica no repositório do projeto)

| Componente        | Caminho                       | Função                        |
|-------------------|-------------------------------|-------------------------------|
| Skills do domínio | .claude/<br>skills/<dominio>/ | Skills específicas do projeto |
| Agents            | .claude/agents/               | Subagentes do projeto         |
| Commands          | .claude/commands/             | Comandos do projeto           |
| Specs             | SPEC*.md                      | Especificações do projeto     |
| Scripts           | scripts/                      | Scripts determinísticos       |
| Templates         | templates/                    | Templates de output           |
| CLAUDE.md         | CLAUDE.md                     | Regras do projeto             |

2. Criando o Repositório Compartilhado

2.1 Estrutura do Repositório token-economy-shared

```
token-economy-shared/  
├── .claude/  
│   ├── skills/  
│   │   ├── lean-ctx/  
│   │   │   └── SKILL.md  
│   │   ├── headroom/  
│   │   │   └── SKILL.md  
│   │   ├── caveman/  
│   │   │   └── SKILL.md  
│   │   ├── rtk-memory/  
│   │   │   └── SKILL.md  
│   │   ├── pre-flight-check/  
│   │   │   └── SKILL.md  
│   │   ├── calcular-gastos-sessao/  
│   │   │   └── SKILL.md  
│   │   ├── fable-method/  
│   │   │   └── SKILL.md  
│   │   ├── fable-judge/  
│   │   │   └── SKILL.md
```

```

|       └─ self-learning/
|           └─ SKILL.md
├─ .code-review-graph/
|   └─ .gitignore
├─ pytest.ini
├─ requirements.txt
├─ setup-links.sh
├─ setup-links.ps1
└─ README.md

```

## 2.2 Comandos para Criar

```

# 1. Criar o repositório compartilhado
mkdir token-economy-shared
cd token-economy-shared
git init

# 2. Copiar as skills globais do projeto atual
cp -r ../proj_fabrica-de-livros/.claude/skills/lean-ctx .claude/skills/
cp -r ../proj_fabrica-de-livros/.claude/skills/headroom .claude/skills/
cp -r ../proj_fabrica-de-livros/.claude/skills/caveman .claude/skills/
cp -r ../proj_fabrica-de-livros/.claude/skills/rtk-memory .claude/skills/
cp -r ../proj_fabrica-de-livros/.claude/skills/pre-flight-check .claude/skills/
cp -r ../proj_fabrica-de-livros/.claude/skills/calcular-gastos-sessao .claude/skills/
cp -r ../proj_fabrica-de-livros/.claude/skills/fable-method .claude/skills/
cp -r ../proj_fabrica-de-livros/.claude/skills/fable-judge .claude/skills/
cp -r ../proj_fabrica-de-livros/.claude/skills/self-learning .claude/skills/

# 3. Copiar configs
cp ../proj_fabrica-de-livros/pytest.ini .
cp ../proj_fabrica-de-livros/requirements.txt .
cp ../proj_fabrica-de-livros/scripts/setup-links.sh .
cp ../proj_fabrica-de-livros/scripts/setup-links.ps1 .

# 4. Criar .gitignore
echo ".code-review-graph/" > .gitignore
echo "__pycache__/" >> .gitignore
echo "*.pyc" >> .gitignore

# 5. Commit e push
git add .
git commit -m "feat: token economy shared infrastructure"

```

```
git remote add origin git@github.com:seu-usuario/token-economy-shared.git
git push -u origin main
```

## 3. Adicionando o Submodule em Projetos Existentes

### 3.1 Comando Único

```
# No diretório do projeto
git submodule add git@github.com:seu-usuario/token-economy-
shared.git .token-economy
```

### 3.2 Estrutura Resultante

```
meu-projeto/
├── .token-economy/                ← SUBMODULE (read-only)
│   ├── .claude/skills/
│   │   ├── lean-ctx/
│   │   ├── headroom/
│   │   ├── caveman/
│   │   └── ...
│   ├── pytest.ini
│   └── requirements.txt
├── .claude/                      ← PROJETO (read-write)
│   ├── skills/
│   │   └── minha-skill/          ← Skills específicas do projeto
│   ├── agents/
│   └── commands/
├── CLAUDE.md
└── scripts/
```

### 3.3 Script de Setup Automatizado

Criar `setup-token-economy.sh` no projeto:



```
#!/bin/bash
# Setup Token Economy via Submodule

SUBMODULE_PATH=".token-economy"

# 1. Adicionar submodule (se não existe)
if [ ! -d "$SUBMODULE_PATH" ]; then
    git submodule add git@github.com:seu-usuario/token-economy-shared.git
    $SUBMODULE_PATH
fi

# 2. Atualizar submodule
git submodule update --init --recursive

# 3. Criar junctions (Windows) ou symlinks (macOS/Linux)
if [[ "$OSTYPE" == "msys" || "$OSTYPE" == "cygwin" ]]; then
    # Windows: junctions
    cmd //c "mklink /J .claude\skills\lean-ctx
$SUBMODULE_PATH\claude\skills\lean-ctx"
    cmd //c "mklink /J .claude\skills\headroom
$SUBMODULE_PATH\claude\skills\headroom"
    cmd //c "mklink /J .claude\skills\caveman
$SUBMODULE_PATH\claude\skills\caveman"
    # ... mais junctions
else
    # macOS/Linux: symlinks
    ln -s "$SUBMODULE_PATH/claude/skills/lean-ctx" ".claude/skills/
lean-ctx"
    ln -s "$SUBMODULE_PATH/claude/skills/headroom" ".claude/skills/
headroom"
    ln -s "$SUBMODULE_PATH/claude/skills/caveman" ".claude/skills/
caveman"
    # ... mais symlinks
fi

# 4. Copiar configs (se não existem)
[ ! -f pytest.ini ] && cp $SUBMODULE_PATH/pytest.ini .
[ ! -f requirements.txt ] && cp $SUBMODULE_PATH/requirements.txt .

# 5. Instalar dependências
pip install -r requirements.txt

echo "Token Economy configurado com sucesso!"
```

## 4. Criando Projeto Novo com Token Economy

---

### 4.1 Template de Projeto

```
# 1. Criar projeto a partir do template
git clone git@github.com:seu-usuario/template-projeto.git meu-projeto
cd meu-projeto

# 2. Renomear
git remote set-url origin git@github.com:seu-usuario/meu-projeto.git

# 3. O submodule já vem configurado no template
git submodule update --init --recursive

# 4. Criar junctions/symlinks
bash setup-token-economy.sh

# 5. Personalizar CLAUDE.md
# Editar CLAUDE.md com as regras específicas do projeto
```

### 4.2 Template `CLAUDE.md` Mínimo

```
# Meu Projeto

## Regras
- PT-BR estrito
- Sem metatexto nos artefatos
- Auto-correção interna antes de entrega

## Stack
- Python 3.11+
- pytest para testes
- Token Economy via `.token-economy/`

## Comandos
- `/executar` – Roda o pipeline principal
- `/validar` – Roda testes e lint
```

## 5. Sincronizando Atualizações

---

### 5.1 Atualizar o Submodule no Projeto

```
# Atualizar para a última versão
cd .token-economy
git pull origin main
cd ..

# Commitar a mudança
git add .token-economy
git commit -m "chore: update token-economy to latest"
git push
```

### 5.2 Propagar Atualização para Todos os Projetos

```
# Em cada projeto que usa o submodule
git submodule update --remote .token-economy
git add .token-economy
git commit -m "chore: sync token-economy"
git push
```

### 5.3 Script de Sincronização em Massa

```
#!/bin/bash
# sync-all-projects.sh
PROJECTS=(
    "/caminho/projeto1"
    "/caminho/projeto2"
    "/caminho/projeto3"
)

for proj in "${PROJECTS[@]"; do
    echo "=== Sincronizando $proj ==="
    cd "$proj"
    git submodule update --remote .token-economy
    git add .token-economy
    git commit -m "chore: sync token-economy" || echo "Nada para commitar"
    git push || echo "Erro no push"
```

```
cd -  
done
```

---

## 6. Submodule vs. Alternativas

---

### 6.1 Git Submodule (Recomendado)

**Prós:** - Versionado junto com o projeto - Atualização controlada ( `git submodule update` ) - Funciona offline após clone - Integração nativa com git

**Contras:** - Requer `git submodule update --init` após clone - Junctions/symlinks precisam ser recriados manualmente

### 6.2 Git Subtree

**Prós:** - Mais simples que submodule - Código fica integrado no repositório

**Contras:** - Duplica o código (não é referência) - Atualização mais trabalhosa

### 6.3 Package Manager (pip/npm)

**Prós:** - Instalação automatizada - Versionamento semântico

**Contras:** - Não funciona bem com arquivos `.md` e configs - Requer registro em registry público/privado

### 6.4 Copiar/Colar (Não Recomendado)

**Prós:** - Simples

**Contras:** - Sem sincronização - Duplicação massiva - Manutenção impossível

---

## 7. Melhores Práticas

---

1. **Versão do Submodule:** usar tag semântica ( `v1.0.0` ) no repositório compartilhado
  2. **Documentação:** manter `README.md` atualizado no repositório compartilhado
  3. **Testes:** rodar testes no repositório compartilhado antes de publicar
  4. **Backup:** sempre ter pelo menos 2 cópias do repositório compartilhado
  5. **Branches:** usar `main` para versão estável e `dev` para experimentação
- 

## 8. Exemplo Prático: Fabrica → Novo Projeto

---

```
# 1. Criar novo projeto
mkdir projeto-novo && cd projeto-novo
git init

# 2. Adicionar token economy
git submodule add git@github.com:seu-usuario/token-economy-
shared.git .token-economy

# 3. Setup
bash .token-economy/setup-token-economy.sh

# 4. Criar CLAUDE.md mínimo
cat > CLAUDE.md << 'EOF'
# Projeto Novo
## Regras
- PT-BR estrito
- Token Economy ativo
EOF

# 5. Commit inicial
git add .
git commit -m "feat: initial setup with token economy"
git remote add origin git@github.com:seu-usuario/projeto-novo.git
git push -u origin main
```

---

## 9. Conclusão

---

Usar Git Submodules para compartilhar a infraestrutura de Token Economy é a abordagem mais escalável e sustentável. Permite:

- **Replicação instantânea** em novos projetos
- **Atualização centralizada** que propaga para todos os projetos
- **Manutenção reduzida** (atualizar uma vez, propagar para N projetos)
- **Consistência** entre projetos da mesma organização

A separação entre Camada Global (submodule) e Camada de Projeto (repositório) garante flexibilidade sem perda de controle.

